Lightweight Sentence Embedding for Similarity Measurement Data Extraction

The goal of this challenge is to develop a lightweight model that converts token-level embeddings of a sentence into a single, combined sentence-level embedding.

Requirements:

Your model should be trained and be compatible with the following versions of dependencies,

- Python 3.12
- torch 2.3.0

Please also note that only 1 CPU core is used for inference. Multithreading will be disabled in torch when your submission is running.

Dataset:

Training dataset is split into four files:

File 1: Each row contains two sentences and label, all separated by tabs. Label shows how well the sentences paired in a row align with each other.

File 2: Tokenization of the first sentences of every row from File 1. Each represented as token-level vectors, with row index correspond to row index in File 1. Every row contains:
- the number of tokens in considered sentence on the first position
- the dimension of token embeddings on the second position (it should be always 768)
- tokens embeddings coordinates in following way: first 768 coordinates form the first embedding, next 768 coordinates form the second embedding and so on.

The number of coordinates in every row is a product of its first two elements (number of tokens times dimension). All numbers are separated by one space character.

File 3: Tokenization of the second sentences of every row from File 1, represented analogously to above.

File 4: Similarity labels (0–5) for each sentence pair, also with corresponding index of the row.

Model inference:

- Model is expected to take a tensor of shape (batch_size, seq_len, 768) as an input with varying value of seq_len, which correspond to the biggest number of tokens among all sentences in considered batch. Remaining tokens are right-padded with zeros.

- The model produces tensor of shape (batch_size, output_dim), with one tensor for each example

from a given batch. `output_dim` can be set to an arbitrary value, but should be always the same.

Model submission:

Create a Python file exposing `get_model()` that returns a torch.nn.Module with shape contract (batch, seq_len, 768) -> (batch, output_dim). The trained model weights must also be uploaded next to the source file, using the same basename and `.bin` extension.

Example (your_model.py):

```
import torch
from torch import nn

class MySentenceEmbedding(nn.Module):
    def __init__(self, input_size=768, output_size=30):
        super().__init__()
        self.fc1 = nn.Linear(input_size, 64)
        self.fc2 = nn.Linear(64, output_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (batch, seq_len, 768) -> out: (batch, output_size)
        # Mean pooling over sequence dimension
        x = torch.mean(x, dim=1)  # (batch, seq_len, 768) -> (batch, 768)
        return self.fc2(torch.relu(self.fc1(x)))

def get_model():
    model = MySentenceEmbedding()
    # Do not load weights here, they will be loaded by by the platform automatically
    return model
```

These two files should uploaded in one .zip file format. For the next submission, edit the previous one by changing .zip file.

Model evaluation:

First, the model converts pairs of sentences from test dataset into pairs of their embeddings. Second, the closeness of obtained pairs and corresponding labels is compared using a built-in algorithm, which return the 'quality score' in range (0, 80). Third, the time of the inference is measured and converted to 'time score' in range (0, 20). Finally, the score is the sum of `quality score` and `time score`, so it is in range (0, 100).